

Vorlesung Rechnerarchitekturen (AIN 2)

Probeklausur

Bearbeitungszeit: 54 Minuten (90 Minuten)
Gesamtpunkte: 45 (75 Punkte)

Name:

Matrikelnummer:

Semester:

Hinweise:

- Bei jeder angegebenen Lösung muss der Lösungsweg nachvollziehbar sein.
- Elektronische Hilfsmittel wie z.B. Taschenrechner, Laptops oder Tablets sind **nicht** erlaubt.
- Erlaubte Hilfsmittel sind Aufschriebe oder Ausdrücke auf Papier.

Aufgabe 1 [Assembler Programmierung] (10 Punkte)

Implementieren Sie die folgende Funktion in MIPS Assembler. Halten Sie sich bei der Implementierung an die MIPS-Konventionen.

```
int f(int A[],int j,int b,int i) {  
    int m=b+8;  
    m=t(I,j)+r(j,i)+m  
    return A[m];  
}
```

Die Funktionen `int t(int,int)` und `int r(int,int)` befinden sich an den Speicheradressen mit Label G und Label H. Sie können davon ausgehen, dass die Funktionen entsprechend der MIPS-Konventionen implementiert sind. (10P)

Aufgabe 2 [Assembler Instruktionen] (9 Punkte)

Die folgenden Tabellen enthalten eine Reihe von Instruktionen, die Sie nacheinander für die ebenfalls in den Tabellen gegebenen Register- und Speicherinhalte ausführen sollen. Tragen Sie die Veränderungen der gelisteten Register- und Speicherinhalte jeweils in den freien Feldern der Tabellen ein.

Hinweise:

- Punkte pro Instruktion wie in der ersten Spalte der Tabelle angegeben.

		Register (Inhalte als Signed Integer)						
		\$s0	\$s1	\$s2	\$t0	\$t1	\$t2	\$sp
P	Instruktionen	1	16	-9	15	-5	-23	0x7FFF AF10
2	lb \$t1, 1(\$s0)					6		
0,5	addi \$s2, \$s1, -9			7				
1	srlv \$t2, \$s1, \$s0						8	
2	lb \$s2, -1(\$sp)			-3				
1	srav \$t1, \$s2, \$s0					-2		
0,5	slti \$s2, \$t1, -3			0				
2	sw \$s1, -7(\$t0)							

Speicherausschnitt					
Adresse (hexadezimal)	Inhalt (unsigned Bytes)		Adresse (hexadezimal)	Inhalt (unsigned Bytes)	
...		Änderung	...	...	Änderung
0x0000 000B	255	0	0x7FFF AF13	0	
0x0000 000A	255	0	0x7FFF AF12	0	
0x0000 0009	4	0	0x7FFF AF11	5	
0x0000 0008	49	16	0x7FFF AF10	16	
0x0000 0007	255		0x7FFF AF0F	253	
0x0000 0006	255		0x7FFF AF0E	255	
0x0000 0005	251		0x7FFF AF0D	255	
0x0000 0004	255		0x7FFF AF0C	255	
0x0000 0003	0		0x7FFF AF0B	48	
0x0000 0002	6		0x7FFF AF0A	0	
0x0000 0001	0		0x7FFF AF09	3	
0x0000 0000	5		0x7FFF AF08	0	

Aufgabe 3 [Maschinensprache] (8 Punkte)

Im Folgenden ist ein Stück Programm-Code sowohl in Assemblersprache als auch in Maschinensprache gegeben. Beide Programm-Codes weisen Lücken auf. Ergänzen Sie diese Lücken.

Hinweis: Punkte pro Instruktion in rechter Spalte.

Verwenden Sie für diese Aufgabe die folgenden OPCODES und FUNCTION Codes:

Code (binär)	000001	000010	000011	000100	000101
Code ist OPCODE von Instruktion	addi	slti	ori	andi	nori
Code ist FUNCT von Instruktion	add	sub	slt	sllv	or
Code (binär)	000110	000111	001000	001001	001010
Code ist OPCODE von Instruktion	lw	sw	beq	bne	j
Code ist FUNCT von Instruktion	and	nor	sra	sll	srl

Verwenden Sie die folgende Registeradressierung:

Nummer	Adresse	Register		Nummer	Adresse	Register		Nummer	Adresse	Register
4	00100	\$t0		8	01000	\$s0		12	01100	\$a0
5	00101	\$t1		9	01001	\$s1		13	01101	\$a1
6	00110	\$t2		10	01010	\$s2		14	01110	\$a2
7	00111	\$t3		11	01011	\$s3		15	01111	\$a3

Füllen Sie die folgenden Lücken in Assembler- bzw. Maschinencode aus:

Speicher- adresse	Maschinenformat								Assembler	P
16	0001	1101	1100	1111	1111	1111	1111	1000	L1: sw \$a3, <u>-8</u> (\$a2)	2
20	0000	0001	0110	0101	0101	0000	0000	0111	L2: <u>nor</u> \$s2, \$s3 \$ <u>t1</u>	1
24	0000	1001	0110	1110	0000	0000	0000	1001	L3: slti \$ <u>a2</u> , \$s3, 9	1
28	0010	0100	1000	1100	1111	1111	1111	1101	L4: bne \$t0, \$a0, L2	1
32	0000	0000	0000	1011	0011	1010	0000	1010	L5: srl \$t3, \$ <u>s3</u> , 8	2
36	0010	1000	0000	0000	0000	0000	0000	0101	L6: j <u>L2</u>	1

Aufgabe 4 [Datenpfad und Steuerwerk] (12 Punkte)

1. In Abbildung 1 sind Datenpfad und Steuerwerk einer Eintakt-Implementierung der MIPS CPU dargestellt. Der Inhalt der Register ist in Tabelle 1 gegeben. (6P)

Einige Daten- und Steuersignale sind mit Nummern bzw. Buchstaben in grau- bzw. blau unterlegten Kreisen versehen. Tragen Sie in Tabelle 4 die Werte ein, die diese Signale bei der Ausführung der Instruktion

Maschinencode:

Binär: 0001 0010 1101 0101 0101 0001 1010 0100

Hex: 12 D5 51 A4

R-Format (binär): 000100 10110 10101 01010 00110 100100

R-Format (dezimal): 4 22 21 10 6 36

I-Format (binär): 000100 10110 10101 0101000110100100

I-Format (dezimal): 4 22 21 20900

J-Format (binär): 000100 10110101010101000110100100

J-Format (dezimal): 4 47534500

im nächsten Takt annehmen. Die Instruktion steht im Speicher an Adresse 73000.

Hinweise:

- Sie können die Signale in Binär-, Dezimal- oder Hexadezimaldarstellung angeben.
- Alle **nicht relevanten** Steuerbefehle werden auf 0 gesetzt.
- Der Ausgang des Datenspeichers ist nur bei Ladebefehlen ungleich 0.
- In Tabelle 2 und Tabelle 3 sind die relevanten ALU Steuerbefehle gegeben.
- Der Code steht noch einmal auf der nächsten Seite

2. Eine MIPS CPU soll genau die folgenden Befehle unterstützen:

add, beq, sw, addi, sub

- a. Wie viele Bits muss dann das ALUOp Steuersignal (Control → ALU Control) mindestens enthalten? Erklären Sie ihre Antwort. (1,5P)

2 Bit: R-Instruktionen werden nicht unterschieden => 4 Befehle.

- b. Wie viele Bits muss der ALU Steuerbefehl (ALU Control → ALU) dazu mindestens enthalten? Erklären Sie ihre Antwort. (1,5P)

1 Bit: 2 Befehle (Addition, Subtraktion)

3. Erweitern Sie Abbildung 1.1 so, dass die Instruktion `sll` unterstützt wird. (3P)

Name:

Matrikelnummer:

0	6	4	9	8	10	12	10	16	10	20	9	24	8	28	10
1	6	5	7	9	10	13	6	17	8	21	8	25	10	29	7
2	9	6	9	10	8	14	10	18	10	22	8	26	7	30	9
3	10	7	6	11	6	15	7	19	8	23	8	27	6	31	9

Tabelle 1: Registerinhalte in Hexadezimaldarstellung

ALU operation	Function	ALU operation	Function
0000	AND	0110	Subtract
0001	OR	0111	set-on-less-than
0010	Add	1100	NOR

Tabelle 2: Übersicht ALU Control

Instruction	ALUOp
lw, sw	00
Beq	01
R-Type	10

Tabelle 3: Übersicht ALUOp-Steuersignale

Daten- und Steuersignale					
1		4		A	
2		5		B	
3		6		memRead	

Tabelle 4: Lösung

Maschinencode:

Binär: 0001 0010 1101 0101 0101 0001 1010 0100

Hex: 12 D5 51 A4

R-Format (binär): 000100 10110 10101 01010 00110 100100

R-Format (dezimal): 4 22 21 10 6 36

I-Format (binär): 000100 10110 10101 0101000110100100

I-Format (dezimal): 4 22 21 20900

J-Format (binär): 000100 10110101010101000110100100

J-Format (dezimal): 4 47534500

Name:

Matrikelnummer:

R-Format (dezimal): 4 22 21 10 6 36

I-Format (dezimal): 4 22 21 20900

beq

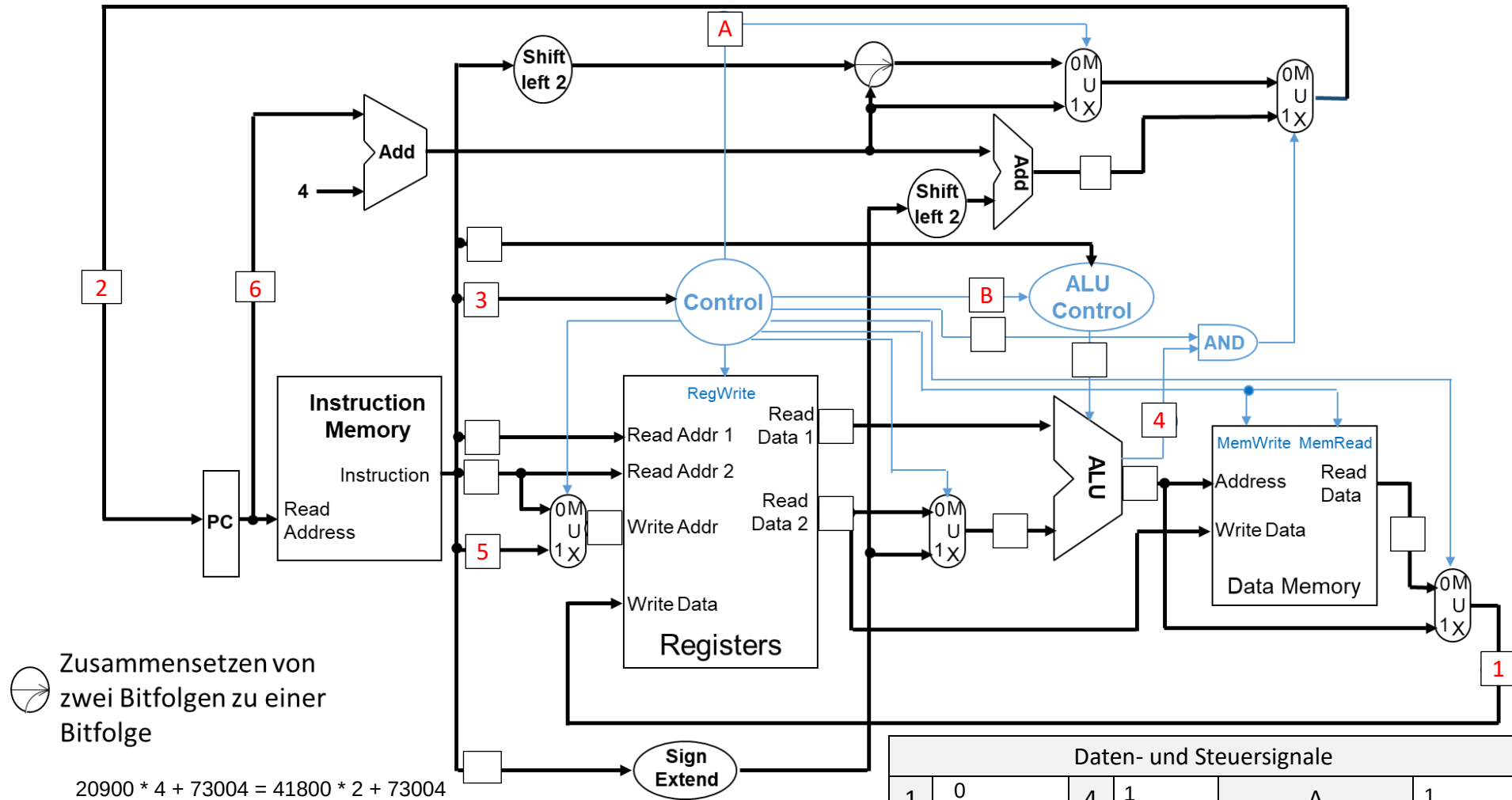


Abbildung 1: Single-Cycle MIPS CPU

Daten- und Steuersignale					
1	0	4	1	A	1
2	156604	5	10	B	01
3	4	6	73000	memRead	0

Aufgabe 5 [Hazards](12 Punkte)

In dieser Aufgabe betrachten wir eine Multi-Cycle CPU mit Forwarding, auf der der Code in Tabelle 5.1 ausgeführt wird. Die CPU besitzt eine Einheit zur Branch-Prediction, so dass der neue Program-Counter bei Branch- und Jump-Instruktionen in der **IF-Stage** bestimmt und geschrieben wird.

- a) Bestimmen Sie alle Data Hazards, die durch Forwarding beseitigt werden. Notieren Sie dazu in der mit „Forwarding“ bezeichneten Spalte das Forwarding in der Form $FW(\$x, R, S)$, wobei R das Pipeline-Register bezeichnet, aus dem der Operand genommen wird und S die Pipeline-Stage ist, in der das Register verwendet wird. (7P)

Beispiel: $FW(\$s0, EX/MEM, EX)$

- b) Tragen Sie in die Spalte „Data Bubbles“ ein, um wie viele Takte sich die Ausführung des Programms bei Verwendung von Forwarding verzögert. (1,5P)
- c) Versuchen Sie, die verbleibenden Bubbles durch Code Scheduling zu vermeiden. (1,5P)

2) Betrachten Sie nun die Control Hazards, die bei der Ausführung des Codes auftreten. Gehen Sie jetzt davon aus, dass der neue Program-Counter bei Branch- und Jump-Instruktionen in der **ID-Stage** bestimmt und geschrieben wird.

- a) Tragen Sie in die Spalte „Control Bubbles“ die Anzahl der Takte ein, die aufgrund von Control Hazards verloren gehen,
- i) wenn bei dem Branch-Befehl in Zeile 6 gesprungen wird und
 - ii) wenn bei dem Branch-Befehl in Zeile 6 nicht gesprungen wird. (2P)
- Verwenden Sie folgendes Format: i) | ii)

Name:

Matrikelnummer:

Tabelle 5.1: Code und Lösungs-Template zu Aufgabe 5

			Forwarding	Data Bubbles	Control Bubbles
1		add \$s1,\$s2,\$t0			
2		lw \$t1,32(\$s1)	FW(\$s1,EX/MEM,EX)		
3		lw \$s3,0(\$s1)	FW(\$s1,MEM/WB,EX)		
4		add \$s2,\$t1,\$t0	FW(\$t1,MEM/WB,EX)		
5		xori \$s1,\$t1,32			
6		bne \$t1,\$s1, L1	FW(\$s1,EX/MEM,ID)	1	
7		j L2			0 1
8	L1:	add \$t0,\$s2,\$s1			1 0
9		add \$s3,\$t1,\$t0	FW(\$t0,EX/MEM,EX)		
10	L2:	sw \$s3,16(\$t0)	FW(\$t0,MEM/WB,EX) FW(\$s3,EX/MEM,EX) FW(\$s3,MEM/WB,MEM)		0 1
			Summe:	1	1 0

Aufgabe 6 [Pipeline] (12 Punkte)

In Abbildung 3.2 ist die MIPS Pipeline mit Fokus auf die EX-Stage dargestellt. Der Code in Abbildung 3.1 wird auf der CPU ausgeführt. Die Register- und Speicherinhalte vor Ausführung von Zeile 1 sind in Tabelle 3.1 gegeben.

- 1) In der Darstellung der Pipeline finden Sie Zahlen in grau-markierten Kreisen. Tragen Sie in Tabelle 2 die entsprechenden Werte ein, wenn die **lw-Instruktion aus Zeile 5** in der EX-Stage ausgeführt wird. Dabei stehen die Zahlen für Pipeline-Register-Inhalte **vor** der Ausführung, Daten- und Steuersignale **während** der Ausführung oder Pipeline-Registerinhalte **nach** der Ausführung der Instruktion in der EX-Stage. Gehen Sie davon aus, dass in dieser Aufgabe **keine Verzögerungen durch Control Hazards** auftreten. (9P)

Hinweise:

- In Abbildung 3.2 finden Sie auch Kopien der Abbildung 3.1 sowie der Tabellen 3.1 und 3.2, um eine einfachere Bearbeitung mit einem Bildschirm zu ermöglichen.
- Die Eingänge der Multiplexer sind von oben nach unten aufsteigend nummeriert. Geben Sie die Steuersignale als Dezimalwerte an.
- Alle **nicht relevanten** Steuerbefehle werden auf 0 gesetzt.
- Sie finden die **lw**-Instruktion in Binär, Dezimal- und Hexadezimalformat auf der nächsten Seite.
- Sie können alle Werte in Binär, Dezimal- oder Hexadezimalformat angeben. Achten Sie aber darauf, dass klar ist, welches Format Sie jeweils verwenden.

- 2) Bestimmen Sie den CPI einer Multi-Cycle CPU mit Branch-Prediction. Gehen Sie von den folgenden Parametern aus: (3P)

- Taktfrequenz: 0.5 GHz
- Anzahl Stages: 16
- CPI aller Instruktionen außer Branches: 3.1
- Branch-Prediction mit Branch-Target-Buffer in der dritten Stage
- Anteil Branch-Instruktionen: 20%
- Anteil der richtigen Vorhersagen der Branch-Prediction: 92%
- Berechnung von Branch-Bedingung und Branch-Adresse erfolgt in der siebten Stage

Name:

Matrikelnummer:

Abbildung 3.1: Code-Ausschnitt

Zeile	Adresse	Instruktion
1	8000	addi \$t3, \$a2, 12
2	8004	lw \$a0, 4(\$a2)
3	8008	addi \$a1, \$s1, 20
4	8012	add \$a0, \$s1, \$a1
5	8016	lw \$a0, 16(\$t3)

Assembler: lw \$a0, 16(\$t3)
Binär (Format): 001011 01011 00100 0000000000010000
Dezimal (Format): 11 11 4 16
Binär (je 4 Bit): 0010 1101 0110 0100 0000 0000 0001 0000
Hexadezimal: 2d 64 00 10

Tabelle 3.1: Register- und Speicherinhalt vor Ausführung von Zeile 1

Register	\$a1	\$a2	\$s1	\$t0
Inhalt (Word)	8	4	20	20
Speicher	8	28	32	36
Inhalt (Word)	16	4	12	4

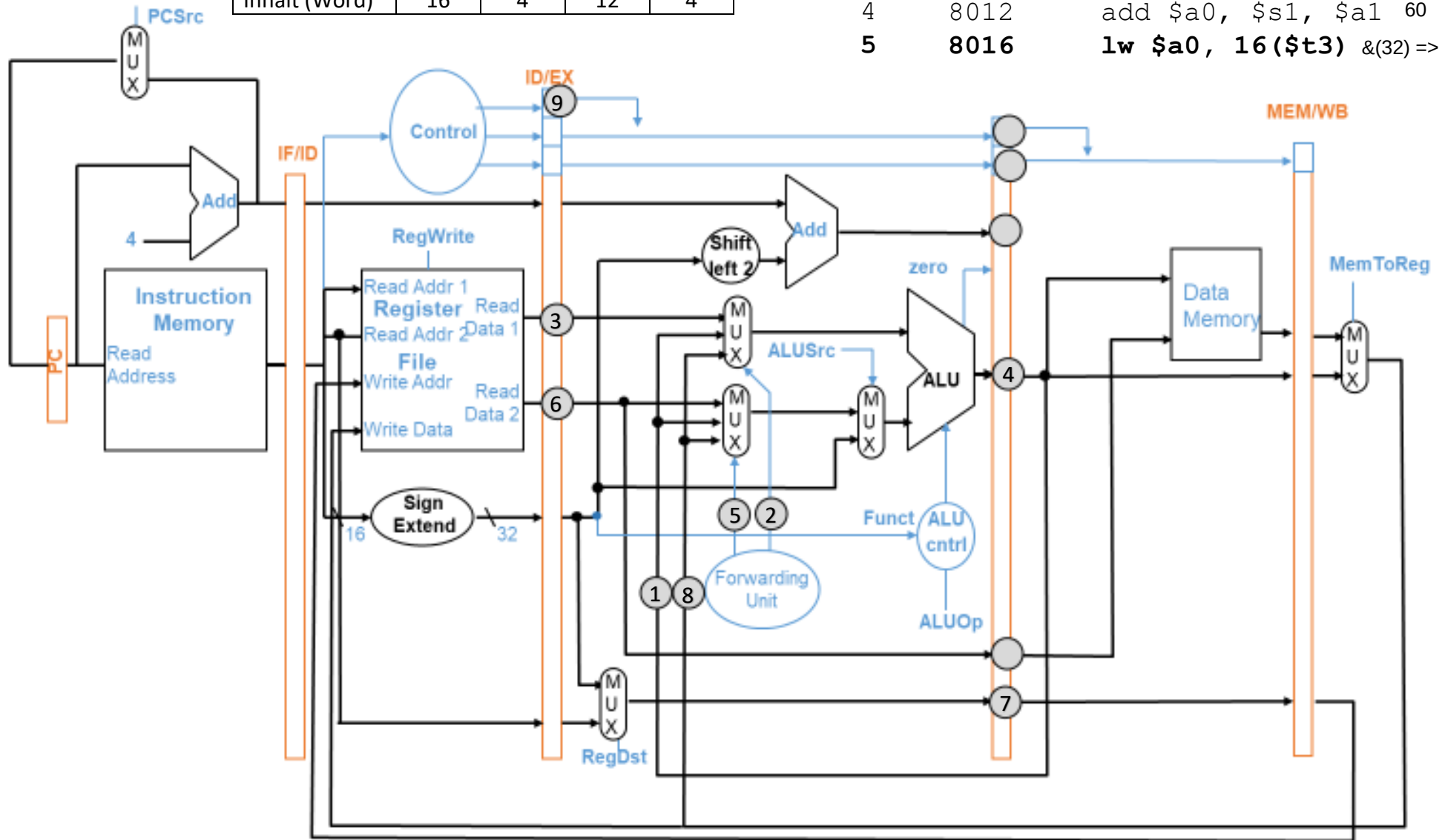
Tabelle 3.2: Registerinhalte, Daten- und Steuersignale bei Ausführung von "lw \$a0, 16(\$t3)" in EX

1		3		5		7	
2		4		6		8	
9							

Name:

Register	\$a1	\$a2	\$s1	\$t0
Inhalt (Word)	8	4	20	20
Speicher	8	28	32	36
Inhalt (Word)	16	4	12	4

Zeile	Adresse	Instruktion
1	8000	addi \$t3, \$a2, 12 16
2	8004	lw \$a0, 4(\$a2) &\$s1
3	8008	addi \$a1, \$s1, 20 40
4	8012	add \$a0, \$s1, \$a1 60
5	8016	lw \$a0, 16(\$t3) &(32) => 12



1	FW(EX/MEM): 60	3	16	5	0	7	32
2	1	4	12	6	32	8	1
9	FW(ID/EX): 40						

Aufgabe 7 [Cache] (12 Punkte)

Ein Prozessor verwendet einen getrennten Daten- und Instruktionscache. Der Daten-Cache wird über eine Speicherhierarchie mit drei Cache-Ebenen realisiert. Auf allen drei Cache-Ebenen wird eine gemeinsame Blockgröße von 8 Bytes verwendet. Eine Übersicht der Cache-Hierarchie finden Sie in der folgenden Tabelle:

Level	Größe	Organisation	Schreib-Strategie
First	512 Bytes	4-way-set associative	Write-Back mit Allocate-on-miss
Second	1024 Bytes	2-way-set associative	
Third	65536 (2^{16}) Bytes	8-way-set associative	

Ein Auszug des Inhalts der Caches ist in der Tabelle auf der nächsten Seite gegeben. Alle Caches verwenden eine LRU-Ersetzungsstrategie. Die Reihenfolge, in der die Blöcke ersetzt werden, wird über die Position des Blocks in der Tabelle vermerkt. Dabei wird der am weitesten rechts stehende Block zuerst ersetzt.

Die folgende Instruktion wird ausgeführt: `lw $t7, 0($sp)`

Vor der Ausführung steht im `$sp`-Register der folgende Wert:

Dezimal: 71620

Binär: 0000 0000 0000 0001 0001 0111 1100 0100

Hexadezimal: 00 01 17 c4

- 1) Tragen Sie in der leeren Tabelle die Änderungen der Cache-Inhalte ein, die durch die Ausführung der „lw“-Instruktion verursacht werden.

Hinweis: Wenn sich lediglich die Position eines Blocks im Set verändert, genügt es lediglich die Tags an der richtigen Position einzutragen. Ein Kopieren der Inhalte ist nicht notwendig. (9P)

- 2) Bestimmen Sie die Anzahl der Takte, die die Ausführung der „lw“-Instruktion benötigt, wenn ein Zugriff auf den First-Level-Cache 3 Takte, ein Zugriff auf den Second-Level-Cache 30 Takte, ein Zugriff auf den Third-Level-Cache 75 Takte und ein Zugriff auf den Hauptspeicher 1500 Takte benötigt. Der Zugriff für das Lesen und Schreiben eines Blocks dauert gleich lang. (3P)

Dauer:

Auszug First-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
5	184	Y	N	47 8D	194	Y	N	55 3D	162	Y	N	15 73	173	Y	N	FE 45
6	993	Y	N	3D 0A	132	Y	N	AC 9E	995	Y	N	38 01	943	N	N	75 6B
7	1002	Y	N	EA 3C	173	Y	N	ED C8	1003	Y	N	60 A0	556	Y	N	97 27
8	898	Y	N	C1 1E	955	Y	N	59 31	947	Y	N	EF 41	957	Y	Y	BC 8F
9	154	Y	N	DA D9	203	Y	N	B5 0E	799	Y	N	46 2D	229	Y	N	9A 0F
10	130	Y	N	37 13	198	Y	N	5F 98	176	Y	N	5E E8	713	Y	N	6A 6B

Auszug Second-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block
21	43	Y	N	FE 45	227	N	N	8E 88
22	248	Y	N	3D 0A	12	N	N	30 78
23	43	Y	N	ED C8	36	Y	N	E0 7C
24	240	Y	N	BC 8F	62	N	N	53 0B
25	57	Y	N	9A 0F	136	Y	N	62 BE

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block
52	60	Y	N	EA E4	238	Y	N	09 00
53	198	Y	N	16 19	251	N	N	C3 E0
54	248	Y	N	38 01	235	Y	N	75 6B
55	250	Y	N	60 A0	39	Y	N	A0 04
56	238	Y	N	59 31	236	Y	Y	EF 41

Auszug Third-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
758	14	Y	N	75 6B	12	Y	Y	23 FA	15	Y	N	47 07	0	Y	N	FC CF	11	Y	N	2C A8	3	Y	N	78 31	8	Y	N	34 3F	16	Y	Y	25 B3
759	14	Y	Y	19 86	8	Y	Y	81 FA	15	Y	Y	7A EA	3	Y	N	7F 70	11	Y	Y	D3 76	0	Y	N	51 AB	12	Y	N	05 E4	2	Y	N	77 5C
760	14	Y	N	3C F9	8	Y	Y	4A 17	15	Y	Y	17 30	3	Y	N	95 B6	12	Y	N	90 C4	11	Y	Y	0C 89	0	N	N	22 C8	6	Y	Y	3A D9
761	11	Y	Y	5F E8	3	Y	Y	7A 2F	0	Y	Y	BD C1	2	Y	Y	08 82	14	Y	N	56 E4	8	Y	N	80 84	12	N	N	10 80	3	Y	N	E9 51
762	2	Y	Y	30 15	11	Y	N	60 AE	14	Y	Y	0E DE	3	Y	Y	73 B7	12	Y	N	55 DE	8	Y	N	1C 8B	0	Y	N	07 6A	15	Y	N	18 1F

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
822	3	Y	Y	8F DB	8	Y	Y	C9 8B	14	Y	N	78 40	0	Y	N	5E 7F	11	Y	Y	CA 90	15	Y	Y	E6 FA	2	N	N	D0 B1	11	Y	Y	99 C2
823	3	Y	N	86 56	15	Y	Y	75 DE	11	Y	Y	12 ED	2	Y	N	8D DD	8	Y	Y	3A 46	0	Y	Y	AA 3A	12	N	Y	6B F8	64	Y	N	F1 0E
824	14	Y	N	EF 41	0	Y	Y	E9 DD	15	Y	Y	08 61	12	Y	Y	6E B5	3	Y	Y	9A 8D	8	Y	Y	C6 E1	11	Y	Y	CF 87	2	N	Y	4E 1D
825	2	Y	Y	C6 51	0	Y	Y	B6 ED	11	Y	N	63 1F	14	Y	N	FE A5	8	Y	N	7C EE	15	Y	N	87 D3	3	Y	Y	36 7C	59	Y	N	C4 06
826	8	Y	Y	6C 7D	14	Y	Y	5F 63	15	Y	N	A7 75	3	Y	Y	66 FE	0	Y	Y	73 50	12	Y	Y	4D 63	11	Y	Y	71 44	2	Y	N	C2 CD

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
983	11	Y	N	E7 AA	2	Y	N	5C 82	8	Y	Y	A1 9E	0	Y	Y	32 46	14	Y	N	C3 DF	12	Y	Y	6F 12	15	N	Y	49 E0	11	Y	Y	FB B8
984	14	Y	N	BC 8F	2	Y	Y	95 0D	11	Y	N	06 43	15	Y	Y	D0 F0	12	Y	Y	60 5F	3	Y	N	18 C7	8	Y	Y	B2 D6	0	N	Y	35 1B
985	3	Y	Y	CF 1D	2	Y	Y	F7 F0	8	Y	N	00 09	0	Y	Y	05 81	12	Y	N	0F 75	11	Y	N	98 0B	15	Y	N	AB 34	14	N	N	5A 4C

Auszug First-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
5																
6																
7																
8	559	1	0	4A 17	898	1	0		955	1	0		947	1	0	
9																
10																

Auszug Second-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block
21								
22								
23								
24								
25								

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block
52								
53								
54								
55								
56	139	1	0	4A 17	238	1	0	

Auszug Third-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
758																												
759																												
760	8	1	1	4A 17	14	1	0																					
761																												
762																												

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
822																												
823																												
824																												
825																												
826																												

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
983																												
984																												
985																												